



BITCOIN PROOF OF WORK



What is “Proof of Work”

- Simply, evidence that a node (miner) exerted considerable effort to reach the result
- In Bitcoin, it is a carefully designed computational challenge to compute a hash that satisfies a condition
- The difficulty of the challenge is “adaptive” and is a function of the state of the network and the time taken to solve the challenge problem

The Challenge

In Bitcoin the Challenge is:

- produce a hash that is *less than* a prespecified target.
- A higher target means it is less difficult to find a hash that is below the target.
- A lower target means it is more difficult to find a hash below the target.
- The target and difficulty are inversely related.

The Target: Better Explained through Python

```
def proof_of_work(header, difficulty_bits):

    # calculate the difficulty target
    target = 2 ** (256-difficulty_bits)

    for nonce in xrange(max_nonce):
        hash_result =
        hashlib.sha256(str(header)+str(nonce)).hexdigest()

        # check if this is a valid result, below the target
        if long(hash_result, 16) < target:
            print "Success with nonce %d" % nonce
            print "Hash is %s" % hash_result
            return (hash_result,nonce)

    print "Failed after %d (max_nonce) tries" % nonce
    return nonce
```

The Target

- As can be seen from the above code, the target is computed using “difficulty bits”

`target = 2 ** (256-difficulty_bits)`

These are usually leading bits in the resulting hash

What to Hash?

- The string to be hashed is a combination of the Block Header + A Nonce
- The Nonce (Number Once) is a random integer variable generated for the purpose

```
hash_result = hashlib.sha256(str(header)
                             +str(nonce)).hexdigest()
```

A Sample Toy Run

```
import hashlib
```

```
text = "I am Mohammed ELAffendi"
```

```
# iterate nonce from 0 to 19
```

```
for nonce in xrange(20):
```

```
# add the nonce to the end of the text
```

```
input = text + str(nonce)
```

```
# calculate the SHA-256 hash of the input
```

```
(text+nonce)
```

```
hash = hashlib.sha256(input).hexdigest()
```

```
# show the input and hash result
```

```
print input, '=>', hash
```

0 => dac6fe1eac2cf9e82dd1e6f71559a6f703c7d3fe42aaa442e8d4741be6d9c6e1
1 => 8e500e40719f0a7146b0077563fddba3d8d2d6d52c1ea3c0d31597a29ce97a8e
2 => 34be29ccb2160c67aab0bf89e165a0528054de15fd263f120c72f84cd6467421
3 => 9b86f88367143dc6c2a15daa52116948754dc35b8638e72e22d423e1e5994c5d
4 => 2c2979a20142c994e65dc764e10f1a9dc8eafbd8444de1687ddc80f9be2c8127
5 => 9ae4c1eb30467eab845cf2c6bfc4ef30c42a0958691a264af1b8188a2f265855
6 => b9708e2725f323ea6f53fb3ad5ed0424a52d6b931d96804a22d8672e90e7af75
7 => 3a9765b015fe8016906e3c83e53cbe4d866473fa4ca02018a15317496b791e94
8 => 317d2e0b38ee2d2e39847734bc5643cf7066cb40d8f03a52700aa26ad7f5d016
9 => 6b3c9be7bcda7c8c7e9d916ef05eb4ef03e7d8d75df94de1d3b18a5326d864a4
10 => 3b47698a1ed5d12ac2ae5eea4a9d2a7ff2220de66138d6f2fa8384873e0e8510
11 => 59c922d29e6db10cfb2194e2ab8e3b2db3aeb658941356bf794eddba67642c30
12 => 4745ba3db7597f450ff253df6c3325ef1e8d037867d8429ed9a6653b16b39310
13 => fd1283170680b60de665a73aad7c9cc1cd6ca9c0f58ce89c4c6a07fef2052420
14 => 550cf003ea24915e05b874cf27d91b114f5f665a785cdee45fdc648e4c0c1cff
15 => 820f993d5d2d0b5e54922349ac3f5df09af12d9062956809f0aed9558969d12b
16 => 410d4d0fa8ae68a6241874bd100a777a591746a4108157b976f32d4717f7f83b
17 => fa8abfa886720aadebd3e5db01ab1e113c84b29918fd5c1dad0c439f89cbe4e0
18 => 6eae62d6232eeb764267a3703b7563cbea1c9f4ad7864a9e28424c38c9fde4f5
19 => 3471761352feaac916ce16eb24c4867bee8762968bd0599aa2d93500be0a581b

Difficulty increases as a function of Leading Zero Bits

```
# difficulty from 0 to 31 bits
for difficulty_bits in range(32):
    difficulty = 2 ** difficulty_bits
    print "Difficulty: %ld (%d bits)" % (difficulty,
difficulty_bits)
    print "Starting search..."
    # checkpoint the current time
    start_time = time.time()

    # make a new block which includes the hash from
the previous block

    # we fake a block of transactions - just a string
    new_block = 'test block with transactions' +
hash_result

    # find a valid nonce for the new block
    (hash_result, nonce) =
proof_of_work(new_block, difficulty_bits)

    # checkpoint how long it took to find a result
    end_time = time.time()
    elapsed_time = end_time - start_time

    print "Elapsed Time: %.4f seconds" %
elapsed_time
```

Time Doubles

- Experiments showed that increasing the difficulty by one bit doubles the computational time

Adjusting Difficulty: Maintain Bitcoin Heart Beat

- Interestingly, Proof of Work has multiple purposes
- One of the major purposes is to maintain Bitcoin Heartbeat: The 10 Minutes interval
- The motivation is to control the money supply and the time for transaction settlement
- For these purposes, Bitcoin employs an a predictive adaptive difficulty adjustment algorithm that predicts the difficulty based on the state of the network and previous experience

Adjusting Difficulty

- Every 2,016 blocks, all nodes retarget (adjust) the Proof-of-Work.
- The adjusting equation measures the time it took to find the last 2,016 blocks and compares that to the expected time of 20,160 minutes (2,016 blocks times the desired 10-minute block interval).

Adjusting Difficulty

- The ratio between the actual timespan and desired timespan is calculated and a proportionate adjustment (up or down) is made to the target.
- If the finding is that blocks mining time is faster than “every 10” minutes, the difficulty increases (target decreases).
- If block discovery is slower than expected, the difficulty decreases (target increases).